Fatec de Mauá · Projeto de Extensão · 3º semestre

Finance Easy

API REST para gestão financeira pessoal

Carlos Henrique de Souza Lima · Guilherme Barbosa da Cruz · Gustavo Silva Ramos · Renan Figueiredo Chipranski

Visão geral

O que é o Finance Easy

Plataforma de controle financeiro pessoal, simples e acessível: registrar receitas e gastos, acompanhar o saldo e planejar metas de economia. O front-end web já existia; neste semestre construímos a **API REST** (Java + Spring Boot) que fornece os serviços de negócio em JSON.

Usuários

Cadastro, listagem, atualização de nome, senha e profissão, e remoção por e-mail

Transações

Renda e gasto com categoria, meio de pagamento e data; saldo calculado automaticamente; consultas filtradas

Metas financeiras

Valor total, valor arrecadado e percentual de conclusão calculado; consultas e ranking

Escopo do MPV

O que entrou e o que ficou de fora

Dentro do escopo

- API REST funcional com 32 endpoints
- Três recursos: usuários, transações e metas
- Banco relacional H2 com carga inicial de dados
- Documentação e testes pelo Swagger UI

Fora do escopo

- Interface web (já existia no front-end)
- Dashboard gráfico
- Exportação do extrato em PDF ou Excel
- Consulta de cotações da B3

Engenharia de Software

Requisitos funcionais e não funcionais

15 requisitos funcionais

- RF01–04: gerir usuários
- RF05–10: cadastrar, consultar, atualizar e remover transações, com consultas especializadas
- RF11–15: cadastrar, consultar, atualizar e remover metas

6 requisitos não funcionais

- Arquitetura REST com troca de dados em JSON
- Java, Spring Boot, JPA/Hibernate e H2
- Endpoints documentados no Swagger UI
- Camadas: controller, service, repository, model
- Banco em memória com carga via import.sql
- Execução local com Maven Wrapper

Engenharia de Software

Principais regras de negócio

E-mail único (RN01)

Não permite dois usuários com o mesmo e-mail: “E-mail já cadastrado”

Saldo automático (RN06)

valorAtual = saldo anterior + valor (renda) ou – valor (gasto)

Recálculo na exclusão (RN07)

Ao remover uma transação, os saldos restantes são recalculados em uma única transação de banco

Categoria compatível (RN08)

Gasto só aceita categorias de gasto; renda só aceita categorias de renda

Percentual da meta (RN10)

(arrecadado ÷ total) × 100, recalculado a cada atualização

Recurso inexistente (RN11)

ID ou e-mail que não existe retorna 404 Not Found

Engenharia de Software

Histórias de usuário e critérios BDD

E01 · Usuários

US01 cadastro
US02 atualização de dados

E02 · Transações

US03 lançamento
US04 consulta filtrada
US05 correção e exclusão

E03 · Metas

US06 cadastro de meta
US07 acompanhamento e comparação

Exemplo (US06): cadastro de meta

Dado um usuário cadastrado, **quando** cria a meta “Celular” de R$ 2.500,00 com R$ 2.200,00 guardados, **então** a meta é salva e mostra 88% de conclusão.

Etapa 2 · Programação

Arquitetura em camadas

Cliente
Swagger UI

→

Controller
endpoints

→

Service
regras de negócio

→

Repository
Spring Data JPA

→

H2
banco em memória

Tecnologias

- Java e Spring Boot 4.1.1 (Web MVC, Data JPA, Validation)
- JPA/Hibernate com banco H2
- springdoc-openapi 3.0.3 (Swagger UI)
- Maven (Maven Wrapper)

Organização do código

- controller: Usuário, Transações e Metas
- service: regras RN01 a RN11
- repository: consultas derivadas, HQL e SQL nativo
- model e projection: entidades, enums e projeções

Etapa 2 · Programação

32 endpoints documentados no Swagger

Usuários

```
POST /usuarios/cadastrar
GET /usuarios/todos
PATCH
/usuarios/{email}/atualizar-nome
DELETE /usuarios/{email}/remover
```

Transações

```
POST /transacoes/cadastrar
GET /transacoes/por-tipo
GET /transacoes/por-periodo
PATCH /transacoes/{id}/atualizar-
categoria
DELETE /transacoes/{id}/remover
```

Metas

```
POST /metas/cadastrar
GET /metas/por-categoria
GET /metas/maior-percentual
PATCH /metas/{id}/valor-
arrecadado
DELETE /metas/{id}
```

Respostas

200 sucesso · **204** transação removida · **400** dados inválidos · **404** não encontrado

Etapa 2 · Validação

25 casos de teste pelo Swagger UI

CT02 · E-mail repetido

Cadastro recusado: “E-mail já cadastrado”

CT09 · Período invertido

Data inicial após a final retorna 400

CT11 · Gasto de R$ 120,00

Saldo de 2.029,25 passa para 1.909,25

CT23 · Meta atualizada

3.000 de 6.000 resulta em 50% de conclusão

Além dos testes manuais, há um teste automatizado (contextLoads) que verifica se o contexto Spring, o H2 e a carga inicial iniciam sem erros.

Etapa 3 · Banco de dados

Modelo de dados e persistência

3 tabelas, relação 1:N

- usuarios
- transacao (usuario_id → usuarios.id)
- metas (usuario_id → usuarios.id)
- Enums gravados como texto

Persistência

- JPA/Hibernate com H2 em memória
- Carga inicial: 7 usuários, 7 transações, 7 metas
- Consultas derivadas, HQL e SQL nativo
- @Transactional na remoção de transação (ACID)

Teste A/B · 1 de 3

Teste A/B da página de login (loginFE.html)

A · Original

Título “Entre ou cadastre-se!”
Login em primeiro plano

B · Variante

Título “Cadastre-se ou entre!”
Cadastro em primeiro plano

50 usuários simulados

25 por grupo, com a mesma composição

15 novos + 10 recorrentes

Novos querem criar conta; recorrentes só querem entrar

Dados simulados: servem como hipótese a validar, não como prova.

Teste A/B · 2 de 3

B converteu mais, mas piorou o login

64% → 80%
Conversão geral

60% → 80%
Novos cadastros

90% → 80%
Login dos recorrentes

28% → 20%
Abandono

32% → 24%
Fricção (cliques errados)

45,6 s → 41,0 s
Tempo médio na página

Da versão A para a versão B.

Teste A/B · 3 de 3

Por que B venceu e o que considerar

Por quê

- Em A, 7 de 15 novos começaram pelo login
- Em B, 4 de 10 recorrentes começaram pelo cadastro
- O formulário do topo vira a ação principal
- Cadastro tem 5 campos; login tem 2

Ressalvas

- Nenhuma diferença é significativa (p ≈ 0,35)
- 25 por grupo; o ideal seriam 100 a 120
- O ganho depende da mistura de perfis

Recomendações

- Deixar o botão “Entrar” bem visível
- Reduzir o cadastro a nome, e-mail e senha
- Validar com cerca de 100 usuários por grupo

Outras métricas também favoreceram B

28% → 20%

Abandono

32% → 24%

Fricção: cliques errados e confusão

45,6 s → 41,0 s

Tempo médio na página

Da versão A para a versão B

Conclusão

Objetivos atingidos e próximos passos

API entregue

- 32 endpoints em 3 recursos
- Regras de negócio e banco relacional
- Documentação e testes no Swagger

Teste A/B

- Variante B teve maior conversão
- Melhor se a maioria for de novos usuários
- Validar com amostra maior

Próximos passos

- Autenticação e criptografia de senhas
- Erros padronizados com códigos HTTP
- Banco persistido em disco
- Integração com o front-end